

Terra das Cinzas: plano técnico

Multiplayer, mercado entre jogadores e conexão com wallet na rede Solana. Visão geral, arquitetura, fases, segurança e pontos de atenção.

1. A decisão mais importante: o que fica on-chain

O jogo hoje roda no navegador, com progresso salvo. Para virar multiplayer com mercado e Solana, a recomendação é um **modelo híbrido**: a jogabilidade em tempo real fica em um servidor do jogo (rápido e barato), e só o que precisa de propriedade verificável vai para a blockchain. Colocar combate, movimento ou cada pickup on-chain deixaria o jogo lento e caro.

Item	Onde fica	Por quê
Posição, combate, mobs, chat	Servidor do jogo (tempo real)	Precisa de dezenas de atualizações por segundo.
Progresso, XP, nível, missões	Banco de dados do jogo	Muda o tempo todo e não precisa de propriedade on-chain.
Ouro do dia a dia	Banco de dados do jogo	Moeda de uso frequente. Pode ter limite de troca.
Itens raros / negociáveis	NFT na carteira do jogador	Propriedade verificável e negociável no mercado.
Moeda premium (opcional)	Token SPL	Só se for realmente necessário. Aumenta a carga jurídica.
Mercado entre jogadores	Programa on-chain (escrow) + servidor	Troca atômica: ou o item e o pagamento trocam juntos, ou nada acontece.

Fluxo recomendado para itens: o item nasce no inventário do jogo (banco de dados). Quando o jogador quer vender ou trocar, ele **transforma o item em NFT** (mint sob demanda) na própria carteira. Ao voltar para o jogo, o NFT é reconhecido e o item aparece no inventário. Isso evita mintar milhões de itens inúteis.

2. Arquitetura sugerida

Camada	Tecnologia sugerida	Função
Cliente	JavaScript atual + Solana Wallet Adapter	Jogo, UI e conexão com a carteira (Phantom, Solflare, Backpack etc.).
Servidor de jogo	Node.js com Colyseus (ou WebSocket próprio)	Salas, canais, sincronização, validação de ações. Servidor autoritativo.
API	Node.js (REST) ou Cloudflare Workers	Login, inventário, mercado, histórico.
Banco de dados	PostgreSQL + Redis	Contas, itens, anúncios do mercado (Postgres); sessões e filas (Redis).
Blockchain	Solana (devnet, depois mainnet-beta)	Propriedade dos NFTs e liquidação das vendas.
NFTs	Metaplex Core (mpl-core)	Padrão recomendado para novos projetos. Conta única, mint mais barato.
Programa do mercado	Anchor (Rust)	Escrow, compra atômica, taxa do jogo, cancelamento.
RPC e indexação	Helius, QuickNode ou similar (com DAS API)	Ler NFTs, receber webhooks de eventos, enviar transações.
Hospedagem	Cloudflare Pages (cliente) + VPS/containers (servidor)	Você já usa pages.dev. O servidor em tempo real precisa de processo persistente.

Observação: os custos e recursos de cada serviço mudam. Confira os preços e limites atuais antes de contratar.

3. Roadmap por fases

Cada fase entrega algo jogável e testável. Não pule direto para a blockchain: valide primeiro o multiplayer e a economia com moeda do jogo.

Fase	Objetivo	Entregas	Pronto quando
0. Base	Preparar o código	Separar lógica de jogo e UI. Definir formato de dados (jogador, item, mob). Criar banco de dados e API de salvar progresso.	Progresso salva no servidor, não só no navegador.
1. Multiplayer	Jogadores se veem	Servidor autoritativo, salas/canais (já existe o menu Canais), movimento e combate sincronizados, chat, grupo.	10+ jogadores no mesmo canal sem travar.
2. Contas e wallet	Login com carteira	Wallet Adapter, login por assinatura (SIWS), vínculo conta-carteira, sessão segura.	Jogador entra só com a carteira, sem senha.
3. Mercado off-chain	Validar a economia	Mercado entre jogadores usando ouro do jogo, taxas, histórico, limites anti-abuso.	Economia estável por algumas semanas de teste.
4. NFTs em devnet	Itens on-chain	Coleção Core, mint sob demanda, sincronizar inventário com a carteira.	Item mintado, visto no jogo e na carteira.
5. Mercado on-chain	Compra e venda real	Programa Anchor de escrow, anúncios, compra atômica, taxa, indexação por webhook.	Venda completa em devnet sem inconsistências.
6. Beta fechado	Mainnet limitada	Auditoria do programa, limites de valor, monitoramento, suporte, revisão jurídica.	Auditoria feita e plano de resposta a incidentes.

4. Multiplayer

- **Servidor autoritativo:** o cliente envia intenções (mover, atacar), o servidor valida e devolve o estado. Nunca confie em valores vindos do navegador (vida, dano, posição, ouro).
- **Opções de framework:** Colyseus (salas e sincronização de estado em Node.js), WebSocket/Socket.IO próprio (mais controle, mais trabalho) ou Nakama (servidor pronto com contas, chat e partidas). Para o nível 300 com canais, comece simples e escale depois.
- **Canais e regiões:** uma sala por canal e região, com limite de jogadores. Envie ao cliente só o que está perto do jogador (interest management) para economizar banda.
- **Suavização:** interpolação do movimento dos outros jogadores e previsão local do próprio jogador, para o jogo parecer fluido mesmo com latência.
- **Persistência:** salve o estado do jogador em intervalos e em eventos importantes (loot raro, venda, level up), de forma transacional.
- **Anti-trapaça básico:** limite de taxa por ação, checagem de velocidade/alcance, logs de ações suspeitas, e banimento por conta e carteira.

5. Login com wallet

O jogador prova que controla a carteira assinando uma mensagem. **Nunca peça chave privada ou frase de recuperação.** O padrão atual é o Sign In With Solana (SIWS), suportado pelo Wallet Standard e pelo Wallet Adapter, que permite conectar e assinar em um só passo. Abaixo, um fluxo simples com assinatura de mensagem (serve como base e funciona na maioria das carteiras).

```

// CLIENTE (navegador)
const { nonce } = await (await fetch("/auth/nonce")).json();
const message = "Entrar em Terra das Cinzas\nNonce: " + nonce;
const sig = await wallet.signMessage(new TextEncoder().encode(message));
await fetch("/auth/verify", {
  method: "POST", headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    publicKey: wallet.publicKey.toBase58(),
    message, signature: Array.from(sig)
  })
});

// SERVIDOR (Node.js)
import nacl from "tweetnacl";
import { PublicKey } from "@solana/web3.js";

const ok = nacl.sign.detached.verify(
  new TextEncoder().encode(message),
  Uint8Array.from(signature),
  new PublicKey(publicKey).toBytes()
);
// Se ok: confirmar que o nonce existe, nao expirou e nunca foi usado,
// apagar o nonce e emitir a sessao (cookie httpOnly ou JWT curto).

```

Regras: o nonce deve ser aleatório, de uso único e expirar em poucos minutos. Inclua o domínio na mensagem para evitar reuso em outros sites. Permita vincular mais de uma carteira à mesma conta só com nova assinatura.

6. Itens como NFT

- **Padrão:** Metaplex Core é o padrão recomendado pela Metaplex para novos projetos de NFT na Solana. Usa uma conta por ativo e custa bem menos para mintar que o Token Metadata (a documentação cita cerca de 0,003 SOL por mint, contra 0,022 SOL). Confira o valor atual antes de planejar custos.
- **Coleção:** uma coleção Core para todos os itens do jogo, com atributos (tipo, raridade, nível, status) nos metadados.
- **Plugins do Core:** permitem royalties aplicados pelo protocolo, atributos e delegados (por exemplo, congelar o item enquanto está listado). Leia a documentação para decidir o desenho do escrow.
- **Compressed NFTs (Bubblegum):** alternativa mais barata para volumes muito grandes, mas exigem RPC com DAS API e têm mais complexidade. Só vale se você for emitir muitos itens.
- **Atributos mutáveis:** evite gravar no NFT coisas que mudam toda hora. Guarde no NFT a identidade e a raridade, e deixe o desgaste, o XP do item etc. no banco de dados do jogo.

Exemplo de metadados de um item

```
{
  "name": "Lamina de Brasa",
  "symbol": "CINZA",
  "description": "Espada forjada nas ruínas de Miraluz.",
  "image": "https://SEU-DOMINIO/assets/nft/lamina-de-brasa.png",
  "attributes": [
    { "trait_type": "Tipo", "value": "Arma" },
    { "trait_type": "Raridade", "value": "Epico" },
    { "trait_type": "Nivel minimo", "value": 20 },
    { "trait_type": "Regiao de origem", "value": "Bosque das Ruínas" }
  ]
}
```

Hospede imagens e JSON em armazenamento estável (Arweave, IPFS com pin ou storage próprio com backup). Se o link morrer, o NFT perde a imagem.

7. Mercado entre jogadores

Faça em duas etapas. Primeiro um mercado **off-chain** com ouro do jogo, para testar preços, taxas e abuso. Depois o mercado **on-chain** para NFTs, com liquidação atômica.

Instrução do programa	O que faz
listar_item	Cria o anúncio com preço e trava o item (escrow ou congelamento via plugin).
cancelar_anuncio	Devolve o controle do item ao vendedor.
comprar_item	Em uma única transação: transfere o pagamento ao vendedor, a taxa ao jogo e o item ao comprador.
atualizar_preco	Altera o preço de um anúncio ativo (opcional).

Pontos de atenção do mercado:

- **Atomicidade:** pagamento e item devem mudar de dono na mesma transação, para ninguém perder um lado.
- **Taxa do jogo:** defina uma taxa sobre cada venda (e royalties, se quiser). Ela também ajuda a controlar a inflação.
- **Moeda de pagamento:** SOL é o mais simples. Stablecoin (como USDC) evita volatilidade de preço, mas adiciona contas de token ao fluxo.
- **Conferir sempre on-chain:** o servidor só entrega o item no jogo depois de confirmar a transação (nível de confirmação adequado). Nunca com base no que o navegador diz.

- **Alternativa:** integrar marketplaces existentes (como Tensor ou Magic Eden) é mais rápido, mas você perde a experiência dentro do jogo e o controle de taxas. Avalie para uma versão inicial.

8. Indexação e sincronização

- Use um provedor de RPC com **webhooks** e **DAS API** para ser avisado de transferências e vendas e para listar os NFTs de uma carteira.

- Mantenha uma tabela espelho no seu banco (anúncios, vendas, dono atual) atualizada por webhook e por um **job de reconciliação** periódico que compara o banco com a blockchain.

- Trate a blockchain como fonte da verdade para propriedade de NFTs e o banco como fonte da verdade para o estado de jogo.

- Planeje falhas: transações podem não confirmar, webhooks podem atrasar ou repetir. Deixe tudo idempotente.

9. Segurança (checklist)

- Nunca guardar chaves privadas ou seed phrases de jogadores. Só chaves públicas.
- A chave que assina mints e administra o programa deve ficar em cofre (HSM/KMS) ou multisig, nunca no código ou no repositório.
- Validar tudo no servidor: preços, quantidades, donos e estados. O cliente é sempre não confiável.
- Programa Anchor: validar todas as contas e assinantes, usar PDAs corretos, proteger contra reentrada, overflow e contas falsas.
- **Auditoria externa** do programa antes de mainnet e limites de valor no beta.
- Rate limit e proteção contra bots em login, mercado e ações de jogo.
- Mecanismo de pausa de emergência (circuit breaker) no mercado.
- Logs, alertas e um plano para incidentes (quem faz o quê se algo der errado).
- Cuidado com phishing: o domínio oficial deve ser claro, e o jogo nunca deve pedir assinatura de transações que o jogador não entenda. Mostre o que cada assinatura faz.
- Fluxo de teste: localnet, depois devnet, depois mainnet-beta. Use faucets de devnet para testar sem custo.

10. Economia do jogo

Economias abertas sofrem com inflação e bots. Defina desde cedo as entradas (faucets) e saídas (sinks) de moeda e item.

- **Entradas:** drops, recompensas de missão, venda para NPCs.
- **Saídas:** taxa do mercado, forja e reparo, custo de mint, consumíveis, cosméticos.
- **Limites:** teto de ouro por dia, nível mínimo para negociar, atraso antes de poder vender um item recém-obtido.
- **Itens de vitrine:** cosméticos (skins, montarias, mascotes) são ótimos para vender sem desequilibrar o combate.

11. Modelo de dados (resumo)

```
players (id, wallet_pubkey, nome, nivel, xp, ouro, criado_em)
items (id, owner_player_id, tipo, raridade, atributos_json,
      nft_mint NULL, estado) -- estado: jogo | mintado | listado
listings (id, item_id, seller_pubkey, preco, moeda, status,
          tx_assinatura, criado_em) -- status: ativo | vendido | cancelado
trades (id, listing_id, buyer_pubkey, preco, taxa, tx_assinatura,
        confirmado_em)
auth_nonces (nonce, expira_em, usado)
```

12. Pontos legais e de conformidade (Brasil)

Não sou advogado. Este é um alerta de temas para levar a um profissional, porque jogos com itens negociáveis por valor real tocam áreas reguladas.

- **Criptoativos:** o Brasil tem marco legal para prestadores de serviços de ativos virtuais (Lei 14.478/2022), regulado pelo Banco Central. Verifique se o seu mercado ou qualquer conversão para reais te enquadra.
- **Tributação:** ganhos de jogadores e receitas do jogo (taxas) têm tratamento fiscal próprio.
- **Sorteios e caixas de loot pagas:** venda de caixas aleatórias cujo conteúdo tem valor de revenda pode ser vista como jogo de azar. Prefira venda direta de itens e evite mecânicas de aposta.
- **LGPD:** carteira vinculada a dados pessoais é dado pessoal. Tenha política de privacidade e base legal.

- **Menores de idade:** defina idade mínima para negociar com valor real.
- **Termos de uso:** deixe claro que itens são ativos de jogo, riscos, ausência de garantia de valor e regras de banimento.
- **Não prometa retorno financeiro.** Comunique o jogo como jogo, não como investimento.

13. Primeiros passos práticos

1. Montar o ambiente: Node.js, Rust e Anchor, Solana CLI e uma carteira de teste na devnet.
2. Criar o servidor de jogo com uma sala (um canal da Região 01) e conectar o cliente atual por WebSocket.
3. Mover o salvamento do progresso para um banco de dados com contas.
4. Adicionar o botão Conectar Wallet e o login por assinatura (seção 5).
5. Criar a coleção Core na devnet e mintar um item de teste (por exemplo, a Lâmina de Brasa).
6. Construir o mercado off-chain com ouro e testar com poucos jogadores antes do on-chain.

14. Documentação para consultar

- Metaplex: developers.metaplex.com (Core, Bubblegum, DAS API)
- Solana Wallet Adapter e Wallet Standard: github.com/anza-xyz/wallet-adapter
- Sign In With Solana: documentação da Phantom e do Wallet Standard
- Anchor (programas em Rust): anchor-lang.com
- Colyseus: docs.colyseus.io
- Documentação da Solana: solana.com/docs

Preços, versões e recursos mudam com frequência. Confirme na documentação oficial antes de implementar.